

Trabalho de Engenharia de Software

Sistema de Pedidos com Design Patterns em Python

Autores: Gabrielle Mascarelo e Henrique Beltrão

Data: Setembro de 2026

Estrutura e Modularização da Solução

Conforme as diretrizes do trabalho, a arquitetura da aplicação foi estruturada de modo a garantir baixo acoplamento, alta coesão e independência de desenvolvimento em paralelo entre os membros da dupla:

- `src/config.py`: Centraliza a configuração global da aplicação através do padrão **Singleton** (`AppConfig`), evitando instâncias redundantes e perda de estado de ambiente.
 - `src/core.py`: Contém o modelo de domínio do pedido (`Order`, `Product`) e o padrão **Builder** (`OrderBuilder`), responsável por construir pedidos de forma fluente e validar regras estruturais (ex.: presença obrigatória de cliente).
 - `src/payments.py`: Isola o subsistema de pagamentos através do padrão **Factory Method** (`PaymentProcessor`, `PixProcessor`, `CreditCardProcessor`, `BoletoProcessor`, `MilhasProcessor`), permitindo a inclusão de novos meios de pagamento sem alterar o fluxo principal de processamento de pedidos.
 - `src/channels.py`: Implementa o padrão **Abstract Factory** (`ChannelFactory`, `Checkout`, `Notification`) para as famílias de produtos dos canais `WEB`, `MOBILE` e `KIOSK`. Utiliza também um **Registry** (`get_channel_factory`, `register_channel_factory`) para permitir o registro de novas fábricas respeitando estritamente o princípio Aberto/Fechado (**OCP**).
 - `src/services.py`: Contém o orquestrador do ciclo de vida dos pedidos (`OrderService`) e o componente de auditoria (`EventLogger`). Adere rigorosamente ao Princípio da Responsabilidade Única (**SRP**), delegando a especialistas a construção de pedidos, cálculo de totais, instanciação de pagamentos e interfaces de canal.
 - `main.py`: Ponto de entrada executável que demonstra a integração completa de ponta a ponta dos canais e processadores de pagamento.
 - `tests/`: Suíte completa de testes unitários e de integração espelhando cada módulo da pasta `src/`.
-

1: Configuração da Aplicação (Singleton)

1.1. Explique por que utilizar `__new__` não impede, por si só, novas execuções de `__init__`.

Em Python, a instanciação de um objeto ocorre em duas etapas distintas e sucessivas:

1. O interpretador invoca o método estático `__new__` da classe, responsável por alocar memória e retornar uma nova instância do objeto.
2. Se o retorno de `__new__` for de fato uma instância daquela classe, o Python imediatamente e de forma automática invoca o método de inicialização `__init__` passando essa mesma instância.

Portanto, mesmo que `__new__` seja sobrescrito para interceptar a criação e retornar sempre a mesma referência já alocada em memória (o **Singleton**), o interpretador continuará chamando `__init__` a cada nova chamada de `AppConfig()`. Isso faria com que todos os atributos fossem reinicializados para seus valores padrão, sobrescrevendo alterações feitas em execuções anteriores. Por esse motivo, é indispensável controlar o estado de

inicialização por meio de uma flag (como `_initialized`), garantindo que o corpo do `__init__` execute apenas na primeira criação.

1.2. Explique como um módulo Python poderia ser utilizado para compartilhar uma configuração sem implementar uma segunda versão do Singleton.

Módulos em Python atuam naturalmente como singletons devido ao mecanismo interno de importação do interpretador. Quando um módulo é importado pela primeira vez, o Python o compila, o executa e armazena o objeto resultante em cache no dicionário global `sys.modules`.

Qualquer importação subsequente daquele módulo (`import config`), em qualquer ponto da aplicação, não reexecuta o arquivo; apenas recupera a mesma referência em cache na memória. Dessa forma, bastaria definir variáveis globais diretamente no arquivo `config.py` (por exemplo: `environment = "production"`). Todas as partes do sistema que importassem o módulo compartilhariam exatamente o mesmo estado, dispensando a escrita de classes com `__new__`.

1.3. Identifique uma possível consequência de possuir um objeto de configuração global compartilhado.

Uma consequência negativa importante é o forte acoplamento e o impacto direto na confiabilidade dos testes unitários. Por se tratar de um estado global mutável compartilhado em memória, alterações feitas por um teste (como mudar `debug = True` ou `environment = "testing"`) persistem para os testes seguintes caso não haja uma rotina explícita de limpeza (**teardown**). Isso gera testes frágeis, intermitentes e com dependências ocultas, dificultando o isolamento dos componentes e a identificação de causas raízes de falhas.

2: Pedido e Builder (Builder)

2.1. Identifique quais componentes da sua implementação correspondem ao Builder e ao objeto construído.

- **Builder:** A classe `OrderBuilder`. Ela encapsula a lógica de montagem passo a passo do pedido, mantém o estado intermediário e fornece métodos com encadeamento fluente (retornando `self`) para configurar os dados do pedido (`set_client`, `add_product`, `set_address`, etc.).
- **Objeto Construído (Produto):** A classe `Order`. É a entidade de domínio definitiva devolvida pelo método `build()`, contendo os dados consolidados e a regra de negócio de cálculo de valor total (`total()`).

2.2. Explique por que seria possível construir o pedido diretamente pelo construtor de Order e qual seria a diferença em relação à solução adotada.

Seria perfeitamente viável instanciar a classe chamando o construtor `Order(...)` diretamente com todos os seus argumentos:

```
Order(client="John", products=[p1, p2], address="Rua X", coupon=None,
      payment_method="Pix", observation=None)
```

A diferença central é que `Order` possui múltiplos atributos opcionais (endereço, cupom, observação, forma de pagamento). Sem o **Builder**, o código cliente é obrigado a lidar com uma longa lista de parâmetros posicionais ou nomeados, passando repetidamente `None` para campos não utilizados, o que degrada a legibilidade e favorece erros de digitação e inversão de argumentos.

Com o **Builder**, obtém-se:

1. **Legibilidade e fluência:** métodos claros e autoexplicativos como `.set_address(...)`.
2. **Flexibilidade:** omissão natural de atributos que não fazem sentido para um determinado pedido.
3. **Validação concentrada:** validações complexas (como a obrigatoriedade do cliente) são executadas exclusivamente no momento da chamada de `.build()`.

3: Pagamento e Factory Method

3.1. Identifique os papéis de: **Creator**, **Concrete Creator**, **Product**, **Concrete Product**.

- **Creator:** `PaymentProcessor`. Classe abstrata que declara o método fábrica abstrato `create_payment()` e define o algoritmo comum de processamento no método `process_order(order)`.
- **Concrete Creator:** `PixProcessor`, `CreditCardProcessor`, `BoletoProcessor` (e posteriormente `MilhasProcessor`). Subclasses que sobrescrevem `create_payment()` para instanciar a forma concreta de pagamento correspondente.
- **Product:** `Payment`. Classe base abstrata que define a interface comum (`pay(amount)`) que todos os produtos de pagamento devem implementar.
- **Concrete Product:** `PixPayment`, `CreditCardPayment`, `BoletoPayment` (e `MilhasPayment`). Implementações concretas que realizam a lógica específica de cada meio de pagamento.

3.2. Explique por que uma função contendo simplesmente uma sequência de `if/elif` escolhendo classes concretas não é, por si só, suficiente para caracterizar o padrão **Factory Method**.

Uma função com blocos `if/elif` caracteriza o padrão **Simple Factory** (ou uma função utilitária de criação), e não o **Factory Method**.

O **Factory Method** baseia-se fundamentalmente em herança e polimorfismo. Sua essência é a inversão de controle: a classe base (**Creator**) implementa a lógica de negócio genérica (`process_order`) e delega a decisão de qual classe instanciar para suas subclasses concretas via método abstrato. Uma cadeia de `if/elif` viola o **Open/Closed Principle (OCP)**, pois a adição de qualquer nova modalidade de pagamento exigiria a modificação direta da função existente.

3.3. Considere que uma nova forma de pagamento seja adicionada posteriormente. Explique quais partes da sua implementação precisariam ser alteradas.

Nenhuma parte do código pré-existente precisaria ser alterada, respeitando integralmente o **OCP**.

Para introduzir um novo meio de pagamento (como demonstrado na Questão 8 com `Milhas`), basta estender o sistema criando duas novas classes:

1. Uma subclasse de `Payment` (ex.: `MilhasPayment`), implementando o método `pay(amount)`.
2. Uma subclasse de `PaymentProcessor` (ex.: `MilhasProcessor`), implementando `create_payment()` para retornar uma instância de `MilhasPayment`.

A classe base `PaymentProcessor` e seu método `process_order()` permanecem intactos, pois operam exclusivamente sobre a abstração `Payment`.

4: Famílias por Canal (Abstract Factory)

4.1. Explique por que **checkout** e **notificação** podem ser considerados uma família de produtos.

No padrão **Abstract Factory**, uma família de produtos é composta por objetos projetados para operarem de maneira conjunta e harmônica sob o mesmo contexto ou restrições de ambiente.

O **Checkout** representa a interface visual e a interação com o cliente durante o fechamento da compra, enquanto a **Notification** é a comunicação de confirmação do pedido. Ambos variam de forma acoplada ao canal de atendimento:

- No canal **WEB**, o **checkout** é renderizado em página/modal HTML e a notificação é despachada via e-mail.
- No canal **MOBILE**, o **checkout** é apresentado em componentes nativos de aplicativo móvel e a notificação é enviada via **Push Notification**.

- No canal **KIOSK**, o checkout é operado em tela **touchscreen** de totem físico e a notificação é emitida através de comprovante térmico impresso no local.

Misturar produtos de canais diferentes (como disparar um **Push Notification** nativo de celular para um consumidor de totem presencial sem identificação móvel) quebra a consistência do sistema. Portanto, formam uma família coesa acoplada pela plataforma.

4.2. Explique qual problema a Abstract Factory resolve nessa situação.

A **Abstract Factory** resolve dois problemas centrais:

1. **Garantia de compatibilidade entre produtos:** Assegura que o cliente (`OrderService`) sempre obtenha produtos compatíveis entre si. Ao delegar a instanciação a uma fábrica específica (ex.: `WebFactory`), é impossível combinar acidentalmente um `WebCheckout` com um `MobileNotification`.
2. **Isolamento da instanciação em relação ao uso:** O código consumidor interage exclusivamente com as interfaces abstratas (`ChannelFactory`, `Checkout`, `Notification`), eliminando condicionais (`if canal == "WEB": ... elif canal == "MOBILE": ...`) espalhadas pelo sistema.

4.3. Explique por que o pagamento não deve fazer parte da fábrica responsável pelo canal.

Canal de venda e meio de pagamento são dimensões de negócio independentes e ortogonais. Um cliente no canal **WEB** pode pagar via PIX, Cartão ou Boleto, assim como um consumidor no canal **MOBILE** ou no **KIOSK** pode optar por qualquer uma dessas formas.

Se a fábrica do canal fosse responsável por criar o objeto de pagamento, ocorreria uma **explosão combinatória** de classes: para N canais e M formas de pagamento, seriam necessárias $N \times M$ fábricas concretas (`WebPixFactory`, `WebCardFactory`, `MobilePixFactory`, etc.). Além disso, violaria o **SRP**, pois a fábrica do canal deve cuidar apenas dos artefatos de interação da plataforma, delegando o processamento financeiro ao **Factory Method**.

5: Seleção de Fábrica e Alteração do Sistema (Registry & OCP)

5.1. Liste os arquivos criados ou alterados para adicionar KIOSK.

Para adicionar o canal KIOSK:

- **Arquivo estendido:** `src/channels.py`. Foram criadas as classes concretas `KioskCheckout`, `KioskNotification` e `KioskFactory`, seguidas pelo registro:

```
register_channel_factory("KIOSK", KioskFactory)
```
- **Arquivos que NÃO precisaram de modificação:** `src/services.py`, a função `get_channel_factory` e o código cliente consumidor (`main.py` apenas selecionou o canal pela chave "KIOSK").

5.2. Explique por que as alterações realizadas são ou não compatíveis com o princípio OCP.

As alterações são **totalmente compatíveis com o Open/Closed Principle (OCP)**.

O sistema é **fechado para modificação** porque a função `get_channel_factory` utiliza um dicionário dinâmico de registro em vez de uma estrutura de `if/elif` rígida. Portanto, para suportar o novo canal, o código interno de busca não precisou de nenhuma alteração. Da mesma forma, o sistema é **aberto para extensão**, pois novos canais podem ser plugados apenas implementando novas classes e invocando `register_channel_factory`.

6: Responsabilidades e Integração (SRP)

6.1. Qual é a responsabilidade principal de cada componente criado?

- AppConfig: Centralizar a configuração global da aplicação via **Singleton**.
- Product: Representar a entidade de item vendido com nome e preço.
- Order: Representar o pedido consolidado e calcular seu valor total (`total()`).
- OrderBuilder: Encapsular a construção passo a passo do pedido com validação obrigatória do cliente.
- Payment / PaymentProcessor: Definir o contrato e o algoritmo genérico de processamento de pagamentos via **Factory Method**.
- Checkout / Notification: Apresentar a interface de checkout e enviar a notificação específica de cada canal.
- ChannelFactory: Fabricar a família de produtos (checkout e notificação) de um canal via **Abstract Factory**.
- `get_channel_factory / register_channel_factory`: Gerenciar o catálogo dinâmico de fábricas de canais (**Registry**).
- EventLogger: Registrar e armazenar os logs de eventos de auditoria de forma isolada.
- OrderService: Orquestrar o fluxo do pedido (checkout → pagamento → notificação → log) sem se acoplar a implementações concretas.

6.2. Escolha três componentes diferentes e indique uma mudança que deveria ficar restrita a cada um deles.

1. Order: Se a fórmula de cálculo do valor total for modificada (ex.: inclusão de impostos ou descontos por volume), apenas o método `Order.total()` é alterado, sem impactar o **builder**, as fábricas ou os serviços.
2. WebCheckout: Se o layout visual ou os dados exibidos na tela web mudarem, apenas a classe `WebCheckout` é modificada, mantendo o `OrderService` e os outros canais intactos.
3. EventLogger: Se os logs passarem a ser gravados em arquivo em disco rotativo ou enviados a um serviço externo de telemetria, apenas o `EventLogger` muda, sem alterar o `OrderService`.

6.3. Identifique uma decisão de projeto da sua solução que poderia ser diferente. Explique qual seria a alternativa e qual seria a consequência dessa mudança.

- **Decisão adotada:** O `OrderService` recebe `channel_factory` e `payment_processor` como argumentos diretamente no método `process_order(order, channel_factory, payment_processor)`.
- **Alternativa:** Injetar essas dependências no construtor da classe (`OrderService(channel_factory, payment_processor)`).
- **Consequência:** A alternativa tornaria cada instância do serviço vinculada a um canal e processador fixos. Embora isso faça sentido para microsserviços dedicados a uma única plataforma, a abordagem adotada torna o `OrderService` sem estado (**stateless**), permitindo que uma única instância processe sequencialmente múltiplos pedidos de diferentes canais e formas de pagamento, promovendo maior reusabilidade.

7: Testes e Alterações (Testes Adicionais)

Foram implementados três testes adicionais em `tests/test_integration.py` que não repetem os exemplos básicos do enunciado:

7.1. Teste relacionado ao Singleton (AppConfig)

- **Comportamento verificado:** Idempotência do método `__init__` e retenção de atributos dinâmicos adicionados em tempo de execução ao obter uma nova referência de `AppConfig()`.
- **Resultado esperado:** O atributo customizado injetado (ex.: `custom_timeout = 45`) permanece acessível na nova referência e os atributos originais (`currency = "BRL"`, etc.) não são restaurados para o padrão.
- **Importância:** Em sistemas modulares, módulos secundários podem registrar configurações adicionais em runtime sem o risco de que uma instanciação subsequente apague o estado acumulado.

7.2. Teste relacionado ao Builder (OrderBuilder)

- **Comportamento verificado:** Isolamento de dados entre pedidos construídos sequencialmente e garantia de que atributos opcionais não informados permaneçam com o valor padrão `None`.
- **Resultado esperado:** O segundo pedido construído com apenas dados obrigatórios mantém `address = None`, `coupon = None` e sua própria lista de produtos, sem contaminação por dados de pedidos anteriores.
- **Importância:** Evita vazamento de dados de clientes anteriores (como cupons promocionais ou endereços residenciais) em fluxos de criação de pedidos em lote.

7.3. Teste relacionado à criação de objetos por uma das fábricas (ChannelFactory)

- **Comportamento verificado:** Registro dinâmico de uma nova fábrica customizada em tempo de execução (`SmartWatchFactory`) e verificação de que os objetos criados implementam os contratos de `Checkout` e `Notification` e operam com o `OrderService`.
- **Resultado esperado:** `get_channel_factory("SMARTWATCH")` instancia a nova fábrica com sucesso e seus produtos completam o ciclo de processamento do pedido sem erros de tipo ou execução.
- **Importância:** Valida o **OCP** e o Princípio de Substituição de Liskov (**LSP**), assegurando que o sistema pode receber extensões de terceiros via **plugins** sem necessidade de recompilação ou alteração do núcleo da aplicação.

8: Situação de Mudança (Nova Forma de Pagamento)

8.1. Quais arquivos foram criados ou modificados?

Foi modificado apenas o arquivo `src/payments.py`, adicionando as classes `MilhasPayment` e `MilhasProcessor`. No arquivo de testes `tests/test_payments.py`, foi acrescentado um caso de teste adicional com a nova modalidade.

8.2. O fluxo principal de processamento precisou ser alterado?

Não. O fluxo principal contido no método `process_order(order)` da classe abstrata `PaymentProcessor` permaneceu absolutamente inalterado: ele continua invocando `self.create_payment()` de forma polimórfica e processando o total do pedido.

8.3. Quais classes existentes precisaram ser modificadas?

Nenhuma classe existente foi modificada. As classes originais (`Payment`, `PixPayment`, `PaymentProcessor`, etc.) permaneceram idênticas. A nova funcionalidade foi incorporada exclusivamente por meio da criação de novas classes derivadas.

8.4. Explique como o Factory Method contribuiu para essa extensão.

O **Factory Method** isolou a responsabilidade de instanciação da responsabilidade de uso. Como a classe base `PaymentProcessor` depende exclusivamente da abstração `Payment` e delega a criação concreta ao método abstrato `create_payment()`, a nova modalidade de milhas pôde ser incorporada apenas estendendo o **Creator** e o **Product**. Isso viabilizou o cumprimento do **OCP**, mantendo o sistema aberto para extensão e fechado para modificação.

8.5. Compare essa alteração com a inclusão do canal KIOSK. Quais são as semelhanças e diferenças arquiteturais entre as duas extensões?

- **Semelhanças:** Ambas as extensões resolvem o problema de adicionar novos comportamentos sem modificar o código existente (**OCP**), delegando a criação de instâncias concretas para fábricas especializadas e operando estritamente sobre interfaces abstratas.
- **Diferenças Arquiteturais:**
 - A extensão de pagamento utiliza o **Factory Method**, cujo foco é instanciar um **único produto** independente (`Payment`).

- A extensão de canal utiliza o **Abstract Factory**, cujo foco é criar uma **família de produtos interdependentes** (Checkout e Notification) que devem variar juntos para garantir consistência visual e operacional. Enquanto adicionar Milhas requer criar um produto e sua fábrica direta, adicionar o canal KIOSK exige criar a fábrica responsável por toda a família de componentes que interagem com o cliente naquele canal específico.